

ASA Connect

Sistemas Operacionais e Computação em Nuvem

Entrega 1 — Parcial (3,0 pontos)

Equipe: Saulo Ribeiro Santos · Gustavo Henrique Da Silva Santos · Luan Rocha da Silva · Thiffany Moraes Vieira da Silva

Curso: Ciência da Computação — Turma 5º Semestre 2026 — FECAP

Repositório: <https://github.com/2026-2-NCC5/Projeto2>

Data: Setembro de 2026

ASA Connect

Sistemas Operacionais e Computação em Nuvem

Entrega 1 - Parcial (3,0 pontos)

- **Equipe:** Saulo Ribeiro Santos, Gustavo Henrique Da Silva Santos, Luan Rocha da Silva e Thiffany Moraes Vieira da Silva
 - **Curso:** Ciência da Computação - Turma 5º Semestre 2026 - FECAP
 - **Repositório:** <https://github.com/2026-2-NCC5/Projeto2>
-

1. Objetivo

Este relatório documenta a configuração, a validação do ambiente Linux inicial e a estruturação da infraestrutura de contêineres/nuvem utilizados como alicerce tecnológico para o **ASA Connect (Agente para o Estudante)**, solução desenvolvida no âmbito do Projeto Interdisciplinar para a **Área do Sucesso Alvarista (ASA)** da **FECAP**.

O propósito central desta etapa é comprovar a capacidade da equipe de provisionar um ambiente operacional seguro, reproduzível e auditável sob a plataforma Linux, aplicando princípios fundamentais de menor privilégio, gestão de permissões, isolamento de processos, observabilidade

de recursos e proteção de segredos antes do deploy completo dos serviços de backend e inferência de IA da Entrega 2.

2. Ambiente Criado

Para este projeto, foi selecionado um ambiente baseado em **Contêineres Docker** utilizando a distribuição **Ubuntu 22.04 LTS (Jammy Jellyfish)** e imagens otimizadas baseadas em **Debian/Python 3.12-slim**.

Justificativa da Escolha:

- **Reprodutibilidade e Portabilidade:** O uso de contêineres Docker assegura paridade rigorosa entre os ambientes de desenvolvimento local dos integrantes da equipe e o ambiente de produção em nuvem (Render / Docker Cloud), eliminando problemas do tipo *"funciona na minha máquina"*.
 - **Eficiência de Recursos:** Ao contrário de máquinas virtuais completas (VMs hipervisoras tipo 2), os contêineres compartilham o kernel do sistema operacional hospedeiro, consumindo uma fração ínfima de memória RAM e CPU, permitindo inicialização instantânea.
 - **Segurança e Isolamento:** Através de namespaces Linux (PID, NET, IPC, MNT, UTS) e cgroups, o processo do backend é isolado do restante do sistema host, facilitando a aplicação de limites estritos de privilégio e recursos.
-

2.1 Criação do Ambiente

Comando utilizado para inicializar e interagir com o contêiner de infraestrutura:

```
docker run -it --name asa-connect-infra -p 8000:8000 ubuntu:22.04 bash
```

Detalhamento dos parâmetros:

- `-it`: Aloca um pseudo-TTY e mantém o STDIN aberto para interação no terminal.
- `--name asa-connect-infra`: Define um identificador único e padronizado para o contêiner.
- `-p 8000:8000`: Mapeia a porta `8000` do host para a porta `8000` interna do contêiner, reservando o canal de comunicação para a API FastAPI.
- `ubuntu:22.04`: Imagem oficial do Ubuntu 22.04 LTS obtida do Docker Hub.

`bash`: Comando de entrada (entrypoint shell*).

[Captura de Tela 1 - Seção 2.1]: Inserir print do terminal executando o comando `docker run` com sucesso e exibindo o prompt `root@:/#`.

3. Configuração do Sistema Operacional e Usuários

3.1 Atualização de Pacotes do Sistema

Para garantir a correção de vulnerabilidades conhecidas e sincronizar a lista de pacotes dos repositórios oficiais:

```
apt update && apt upgrade -y
```

[Captura de Tela 2 - Seção 3.1]: Inserir print do terminal demonstrando a execução do `apt update && apt upgrade -y` sem erros e finalizado com sucesso.

3.2 Criação de Usuário Não-Root (Princípio do Menor Privilégio - PoLP)

A execução de serviços corporativos sob o usuário `root` representa grave risco de segurança. Por isso, foi provisionado um usuário exclusivo para a aplicação denominado `asaadmin`:

```
adduser asaadmin  
usermod -aG sudo asaadmin
```

Propósito e Boas Práticas:

- O usuário `asaadmin` possui permissões restritas ao seu diretório home e não tem autorização para alterar binários do sistema sem elevação explícita via `sudo`.
- Isso impede que eventuais falhas na camada da aplicação resultem em comprometimento do kernel ou do hospedeiro.

[Captura de Tela 3 - Seção 3.2]: Inserir print do terminal mostrando os comandos `adduser asaadmin` e `usermod -aG sudo asaadmin` sendo executados.

4. Rede

A conectividade e o mapeamento de portas foram verificados por meio dos utilitários de rede do Linux:

```
ip a  
hostname -I
```

Análise das Interfaces de Rede:

- `lo` (Loopback - `127.0.0.1/8`): Utilizada para comunicações internas locais.

- `eth0` (`172.17.0.x/16`): Interface virtual atribuída pela bridge padrão da engine Docker, permitindo roteamento seguro entre contêineres e saída externa para APIs da FECAP/OpenAI/Gemini.
- **Porta Exposta** (`8000/TCP`): A porta `8000` está vinculada para escuta da API RESTful FastAPI (documentada em `/docs` via OpenAPI/Swagger) e comunicação com o frontend Flutter.

[Captura de Tela 4 - Seção 4]: *Inserir print do terminal exibindo a saída dos comandos `ip a` e `hostname -I` no contêiner.*

5. Permissões e Segurança de Arquivos

Para validar a integridade e proteção das pastas do sistema, foram executados os seguintes comandos:

```
whoami
id asaadmin

ls -la /home

chmod 750 /home/asaadmin
```

Justificativa da Máscara de Permissão `750` (`rwxr-x---`):

- **Proprietário** (`asaadmin` - `7` / `rw`): Acesso total para leitura, gravação e execução de scripts da aplicação e base de conhecimento.
- **Grupo** (`asaadmin` - `5` / `r-x`): Usuários pertencentes ao mesmo grupo administrativo podem navegar e consultar arquivos de apoio, sem permissão de alteração indevida.

Outros (Others* - `0` / `---`): Nenhum outro usuário ou processo não autorizado no sistema possui permissão de leitura, escrita ou execução neste diretório. Isso protege diretamente os artefatos de dados e configurações contra acessos indevidos, em conformidade com as diretrizes da LGPD.

[Captura de Tela 5 - Seção 5]: *Inserir print do terminal mostrando o resultado do comando `ls -la /home` com as permissões `drwxr-x---` atribuídas a `/home/asaadmin`.*

6. Processos, Logs e Consumo de Recursos

6.1 Processos em Execução

Inspeção da tabela de processos isolada pelo PID Namespace do Linux:

```
ps aux
```

- Identificação do processo principal (PID 1) e validação de que não existem processos órfãos ou serviços desnecessários em execução em segundo plano.

[Captura de Tela 6 - Seção 6.1]: *Inserir print da saída do comando* `ps aux`.

6.2 Logs do Sistema e Auditoria

Verificação dos arquivos de rastreabilidade de eventos e instalações:

```
ls -la /var/log  
tail -n 30 /var/log/dpkg.log
```

- O diretório `/var/log` armazena o histórico de pacotes instalados (`dpkg.log`) e acessos (`auth.log`), essenciais para auditoria técnica e atendimento aos requisitos RF10 do projeto.

[Captura de Tela 7 - Seção 6.2]: *Inserir print do terminal exibindo a listagem de* `/var/log` *e as últimas linhas do log* `dpkg.log`.

6.3 Consumo de CPU, Memória e Disco

Monitoramento do estado de utilização de recursos computacionais:

```
top -b -n 1  
free -h  
  
df -h
```

- **CPU:** Utilização próxima a 0% em estado ocioso.
- **Memória RAM:** Consumo reduzido inferior a 100 MB na camada base, comprovando o baixo footprint da infraestrutura escolhida.

Disco: Partição raiz alocada eficientemente através do sistema de arquivos em camadas (overlay2* do Docker).

[Captura de Tela 8 - Seção 6.3]: *Inserir print do terminal com os comandos* `top -b -n 1`, `free -h` e `df -h`.

7. Diagrama da Infraestrutura

Abaixo está a representação arquitetural do ecossistema do **ASA Connect**, ilustrando o fluxo de requisições e a integração dos serviços:

```

graph TD
    subgraph Cliente_Mobile ["📱 Camada de Apresentação (Mobile / Web)"]

        User["Estudante / Atendente ASA"]

        App["App Flutter (Dart)\nInterface Responsiva & Acessível"]

        User -->|Interage| App

    end

    subgraph Nuvem_Host ["☁️ Infraestrutura de Nuvem & Contêineres (Linux)"]

        subgraph Docker_Env ["Docker Container (FastAPI - Porta 8000)"]

            API["API Gateway / Routers\n(FastAPI / Python3.12)"]

            Auth["Middleware de Autenticação\n(JWT & Menor Privilégio)"]

            RAG["Pipeline de IA RAG\n(TF-IDF + Cosine Similarity)"]

            Audit["Módulo de Auditoria & Logs\n(/var/log & DB Logs)"]

            API --> Auth

            Auth --> RAG

            API --> Audit

        end

    end

    subgraph Dados_Gov ["💾 Camada de Persistência & Conhecimento"]

        KB["Base de Conhecimento Oficial ASA\n(Markdown / Chunks Indexados)"]

        DB["Banco de Dados Relacional\n(PostgreSQL / SQLite)"]

    end

```

```

    subgraph Provedor_IA ["🌸 Serviço Externo de IA"]

        Gemini["Google Gemini LLM API\n(Geração Sintetizada & Respostas)"]

    end

end

App -->|HTTPS / JSON Token JWT| API

RAG -->|Recupera Contexto & Fontes| KB

API -->|Persiste Histórico & Usuários| DB

RAG -->|Prompt Enriquecido + Grounding| Gemini

Gemini -->|Resposta Estruturada com Fontes| RAG

```

Descrição do Fluxo:

- O usuário acessa a aplicação **Flutter** autenticado via token **JWT**.
- A requisição HTTP REST trafega de forma criptografada para o contêiner da **API FastAPI** (porta 8000).
- O serviço de IA do ASA Connect realiza busca vetorial por similaridade de cosseno nos documentos oficiais do ASA (cesta de serviços, manuais, bolsas e regulamentos).
- A resposta sintetizada é gerada com citação de fontes, data de atualização, links oficiais e indicação de atendimento humano quando aplicável.
- Cada evento é registrado nos logs estruturados e no banco de dados para fins de conformidade e auditoria.

8. Política Básica de Segredos

Para atender rigorosamente aos Requisitos Não Funcionais de **Segurança, Privacidade e LGPD**, foram estabelecidas as seguintes diretrizes:

- **Vedação de Credenciais no Versionamento:** Nenhuma chave de API privada (`GEMINI_API_KEY`), segredo de assinatura criptográfica (`JWT_SECRET_KEY`) ou senha de banco de dados (`POSTGRES_PASSWORD`) é comitada no Git.

- **Documentação por `.env.example`**: O repositório disponibiliza um arquivo modelo (`.env.example`) com nomes de variáveis e valores fictícios, documentando a especificação do ambiente sem expor dados confidenciais.
- **Proteção via `.gitignore`**: O arquivo `.gitignore` na raiz do repositório inclui regras explícitas (`.env` , `.db` , `.pem` , `*.key` , `__pycache__`) para impedir o envio acidental de dados sensíveis.
- **Isolamento de Execução**: Em ambiente de produção e testes, os serviços rodam sob o usuário `asaadmin` , respeitando o princípio do menor privilégio.

[Captura de Tela 9 - Seção 8]: Inserir print do arquivo `.gitignore` e do `.env.example` abertos no VS Code ou no repositório GitHub.

9. Evidências de Execução

As configurações e arquivos demonstrados neste relatório encontram-se estruturados no repositório oficial do projeto:

- **Dockerfile e Docker Compose**: Localizados em `/src/backend/Dockerfile` e `/src/backend/docker-compose.yml` .
- **Arquivo de Variáveis de Ambiente**: Localizado em `/src/backend/.env.example` .
- **Regras de Proteção de Segredos**: Definidas em `/.gitignore` .
- **Módulo do Agente e Knowledge Base**: Localizados em `/src/backend/app/agent/` e `/src/backend/knowledge_base/` .

10. Repositório da Infraestrutura

O código-fonte completo, scripts de automação, documentação e infraestrutura do projeto estão disponíveis no repositório oficial:

🔗 **Repositório GitHub:** <https://github.com/2026-2-NCC5/Projeto2>

11. Considerações Finais

Nesta primeira etapa, foi concluída com êxito a estruturação e a validação do ambiente operacional Linux, assegurando um ecossistema seguro, eficiente e aderente às boas práticas de menor privilégio, permissões Unix (`750`), gerenciamento de processos e governança de segredos.

Para a **Entrega 2 (Final)**, o planejamento contempla a implantação em nuvem pública (Render/AWS), orquestração da API com banco PostgreSQL via Docker Compose, monitoramento de métricas em tempo real (latência, taxa de erro, consumo de memória), execução de testes de carga simulando múltiplos acessos concorrentes de estudantes e integração total de ponta a ponta com a aplicação mobile Flutter.

